

1 验证计划说明.....	1
2 定向测试.....	4
2.1 RISC-V CPU 核心框架与基础功能.....	4
2.1.1 复位与启动时序.....	4
2.1.2 PC 跳转逻辑.....	5
2.1.3 指令闭环逻辑.....	5
2.1.4 取指译码逻辑.....	6
2.1.5 ALU 计算逻辑.....	6
2.1.6 RF 读写逻辑.....	7
2.1.7 立即数拓展逻辑.....	7
2.2 R-TYPE 指令测试.....	8
2.3 I-TYPE 指令测试.....	12
2.4 SW 指令测试.....	14
2.5 LW 指令测试.....	15
2.6 B-TYPE 指令测试.....	16
2.7 JAL 指令测试.....	18
2.8 JALR 指令测试.....	19

1 验证计划说明

序号	测试用例名称	测试说明	预期测试结果
CPU 基础框架			
1	复位与启动时序	检查 Reset 信号撤销后，系统是否从预设状态正常启动。	复位期间 PC 指向起始地址 0x2000。
2	PC 跳转逻辑	验证顺序执行 (PC+4) 与分支/跳转目标地址的切换逻辑。	顺序执行时 PC 逐次加 4；跳转有效时，NPC 准确切换至目标分支地址。
3	指令闭环逻辑	验证单条指令从进入 CPU 到结果写回的过程是否完整。	指令执行各阶段 (IF-DAE-WB) 状态机转换顺畅。
4	取指译码逻辑	验证指令存储器接口读取与译码电路的实时性。	指令存储器输出 (in_ins) 随 PC 更新，译码逻辑作为组合逻辑，其输出控制信号随指令寄存器 (out_ins) 的更新而同步产生，并驱动后续数据通路
5	ALU 计算逻辑	验证算术逻辑单元对不同运算类型 (加、减、逻辑、移位) 的覆盖。	ALU 能根据译码出的操作码切换计算模式，其输出端 ALU_result 能够随输入端操作数的变化而产生正确响应。
6	RF 读写逻辑	验证通用寄存器组 (RF) 的寻址读取与写回使能逻辑。	系统能准确从指定地址读出寄存器原值，并在 WB 阶段通过 RFWrite 信号控制计算结果正确存入目标 rd。
7	立即数扩展逻辑	验证指令字中立即数部分的提取与位宽扩展功能。	针对含有立即数的指令 (如 I-Type)，ImmGen 模块能根据指令类型正确提取对应位段并完成 32 位符号扩展。

指令测试			
1	R-Type 指令测试	配置 ADD、SUB、OR、AND、XOR、SLL、SRL、SRA 等算术逻辑指令。	(1) 准确读出 rs1 与 rs2 寄存器中的原始操作数； (2) ALU 运算结果与指令功能（如 ADD/SUB）匹配； (3) 在 WB 阶段将结果准确存入 rd 地址。
2	I-Type 指令测试	验证带有立即数的算术指令（如 ADDI、ORI 等）的符号/无符号扩展逻辑及运算。	(1) 准确读出 rs1 寄存器值与扩展后的 32 位立即数； (2) ALU 完成混合运算且结果正确； (3) 运算结果在写回阶段成功存入目标寄存器 rd。
3	LW (Load Word) 指令测试	验证从内存加载数据到寄存器的通路。	(1) ALU 计算“rs1+offset”得到正确的访存地址；(2) 内存读使能有效，从 DM 读出目标数据；(3) 数据在 WB 阶段成功写入寄存器 rd。
4	SW (Store Word) 指令测试	验证从寄存器存储数据到内存的通路。	(1) ALU 计算“rs1+offset”得到正确的写入地址；(2) 内存写使能 MemWrite 有效；(3) 寄存器 rs2 的值在时钟沿准确写入 DM 对应地址。
5	分支指令测试	验证基于比较结果的条件跳转指令 BNE, BEQ。	(1) ALU 准确产出 Zero 标志位； (2) 若条件成立，PC 更新为当前 PC+偏移量；若不成立，PC 顺序执行 +4。
6	无条件跳转	验证基于 PC 偏移的直接跳转及	(1) PC 无条件更新为目标跳转

	(JAL)测试	链接保存。	地址； (2) 返回地址（当前 PC+4） 同步写入目标寄存器以备返回。
7	无条件跳转 (JALR)测试	验证基址加立即数的地址计算与 跳转逻辑。	(1) 立即数扩展模块输出正确 位宽的偏移量； (2) ALU 将 rs1 原值与该立即 数相加得到目标地址； (3) PC 跳转至计算结果地址， 并同步写回返回地址(PC+4)到 目标寄存器 rd。

表 1 验证计划说明

2 定向测试

2.1 RISC-V CPU 核心框架与基础功能

本阶段验证的主要目标是确保 CPU 的基础设施能够正常运转，具体包括：

- 复位与启动时序：验证硬件复位信号撤销后，程序计数器（PC）能准确初始化为 0x2000。
- PC 跳转逻辑：确保 PC 地址递增逻辑（PC+4）在顺序执行时保持稳定；同时验证 PC 在接收到跳转使能信号时，能瞬间打断连续性并准确指向目标分支地址。
- 指令闭环逻辑：验证从“取指-译码/执行-写回”的单一数据通路是否打通，确保状态机（IF-DAE-WB）切换顺畅且无挂死现象。
- 取指译码逻辑：验证指令存储器接口在 PC 驱动下的读取响应，以及译码电路（组合逻辑）对指令码拆解的实时性与准确性。
- ALU 计算逻辑：验证运算单元对基础算术/逻辑控制信号（ALUOp）的响应能力，确保其输出端能随输入操作数的变化而产生预期的逻辑结果。
- RF 读写逻辑：验证通用寄存器组（RF）的双端口读取与单端口写入功能，确保 rs1/rs2 地址寻址无误，且写回数据在 WB 阶段准确锁存至 rd 寄存器。
- 立即数扩展逻辑：验证 EXT 模块能根据输入立即数类型准确扩展至 32 位。

本阶段侧重于 CPU 内部通路验证。未提及的数据存储器（DM）读写逻辑及复杂的地址计算，将会在后续 LW/SW 相关指令测试中一并展示。

2.1.1 复位与启动时序

如图 2-1-1 所示，本实验对 CPU 的初始启动逻辑进行了验证。

复位响应：当硬件复位信号 `rst` 为高电平（1）时，程序计数器 `PC` 立即进入初始化状态，其值被准确强制置为 0x00002000。

启动时序：在复位信号撤销（变更为 0）后的第一个有效时钟上升沿，CPU 能够稳定从该初始地址（0x00002000）开始执行程序。

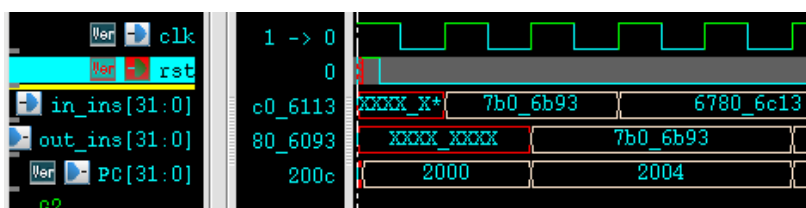


图 2-1-1 复位与启动时序

2.1.2 PC 跳转逻辑

如图 2-1-2 所示，本节验证了程序计数器（PC）的更新机制与跳转使能控制逻辑：

更新触发机制：当控制信号 `PCWrite` 为高电平时，PC 会在下一个时钟上升沿被更新为 NPC 的值。

时序领先特性：在逻辑链路上，NPC 的值始终领先 PC 一个时钟周期。这是由于 NPC 属于组合逻辑输出，实时反映了当前周期根据跳转需求计算出的目标地址。

跳转路径选择：NPC 的具体数值取决于 `NPCOp` 信号的编码。该信号决定了 NPC 是执行顺序执行的 `PC + 4`，还是跳转至分支目标地址。

使能动态控制：`PCWrite` 信号的拉高时机与持续周期并非固定，而是根据当前执行指令的类型动态调整，以确保取指阶段与指令执行流严格同步。

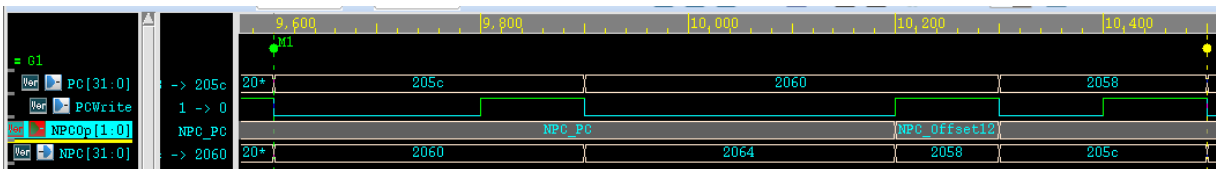


图 2-1-2 PC 跳转逻辑

2.1.3 指令闭环逻辑

如图 2-1-3 与 图 2-1-4 所示，本节对 CPU 的指令执行周期及各阶段的逻辑闭环进行了验证：

常规指令周期划分：由图 2-1-3 可见，除系统启动后的第一个指令周期包含 IT（初始化）状态外，后续所有常规指令的执行均严格遵循 IF（取指）、DAE（译码/执行）与 WB（写回）三个标准阶段。这一三阶段闭环确保了指令从存储器提取到寄存器结果更新的完整性。

访存指令的特殊阶段：如图 2-1-4 所示，针对 LW（加载）和 SW（存储）这两类访存指令，程序会有额外的 MEM（内存访问）阶段。

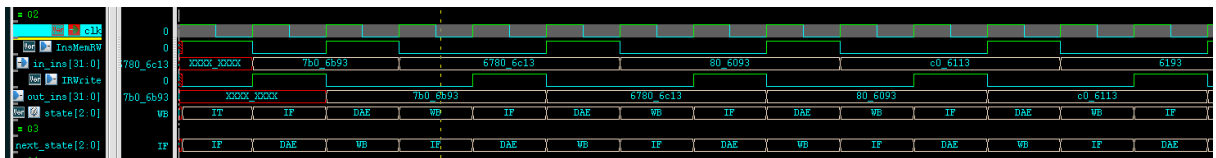


图 2-1-3 指令闭环状态机



图 2-1-4 LW 指令状态机逻辑

2.1.4 取指译码逻辑

如图 2-1-5 所示，本节对指令存储器的访问以及指令寄存器（IR）的锁存逻辑进行了验证：

取指同步更新：当指令存储器读使能信号 `InsMemRW` 拉高后，输入指令码 `in_ins` 会在随后的时钟上升沿完成采样与更新。

指令锁存机制：`IRWrite` 信号控制指令寄存器的锁存。在其拉高后的下一个时钟上升沿，指令码由 `in_ins` 传输至 `out_ins`（即指令寄存器输出端），完成指令的阶段同步。

组合逻辑译码：由于译码电路采用组合逻辑实现，其控制信号的解析与 `out_ins` 的更新几乎同步完成。实验观测显示，一旦指令寄存器输出稳定，译码器即刻产出相应的操作码（`opcode`）与控制字段。

时序调度策略：所有指令的 `IRWrite` 信号均固定在 IF（取指）阶段拉高以确保指令载入。而 `InsMemRW` 信号的使能时机则展现出灵活性，会根据指令类型的差异在特定的阶段进行动态调整。

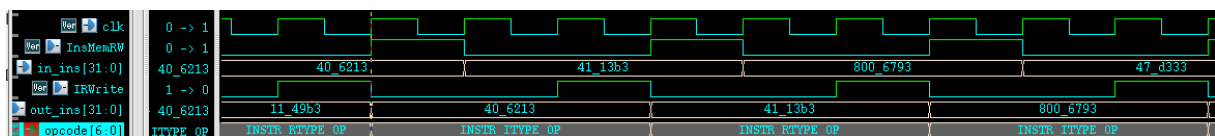


图 2-1-5 取指译码逻辑

2.1.5 ALU 计算逻辑

如图 2-1-6 所示，本节验证了算术逻辑单元（ALU）的操作数获取与运算执行过程：

操作数锁存：在译码阶段确定执行路径后，控制单元产生 `ALUSrcA` 和 `ALUSrcB` 选择信号。寄存器 A 和 B 随后获得相关的操作数值（本仿真中对应寄存器 `x1` 与 `x2` 的值），为后续运算提供稳定的输入源。

计算执行：ALU 根据当前的 `ALUOp` 操作码类型对输入端数据进行算术或逻辑运算，计算结果实时输出至 `ALU_result` 信号线上。

结果缓冲：为了配合多周期时序步进，`ALU_result` 会经过一级寄存器（`Flopr`）进行锁存，保存为同步后的 `ALU_result_r`，以便在后续的周期（如访存或写回阶段）中使用该运算结果。

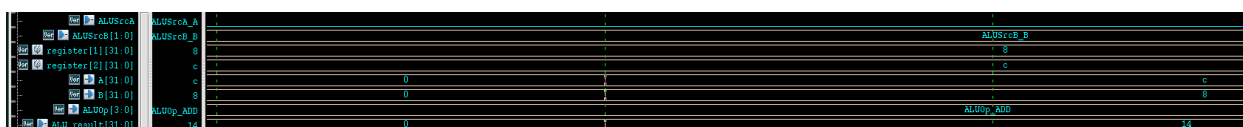


图 2-1-6 ALU 计算逻辑

2.1.6 RF 读写逻辑

如图 2-1-7 所示，本节验证了通用寄存器组（RF）的寻址读取与数据写回控制逻辑：

异步读取过程：在译码阶段，系统根据指令字段确定读取地址 RR1 和 RR2，并从寄存器堆中即时读出对应操作数，分别存入中间寄存器 RD1 和 RD2（对应图中 x1 和 x2 的值）。

写回路径选择：写入数据 WD 的来源由控制信号 WDSel 进行选择。在默认状态下，WDSel 选择 WD_FromALU 路径。本仿真案例展示了执行 $x1 + x2$ 的运算结果作为 WD 输入，准备写入目标寄存器。

同步写入控制：确定写入地址 WR（本例为 x11 的地址）后，当控制信号 RFWrite 在特定周期拉高，数据 WD 会在时钟上升沿准确写入 WR 所指向的寄存器中，完成数据的闭环更新。

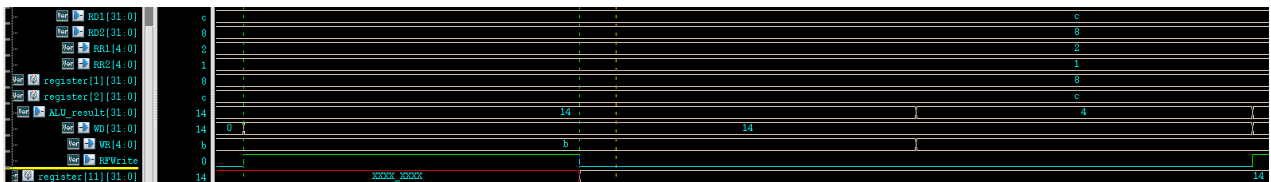


图 2-1-7 RF 读写逻辑

2.1.7 立即数拓展逻辑

如图 2-1-8 所示，本节验证了扩展模块（EX）对指令字中立即数部分的位宽转换与符号处理逻辑：

输入捕获：EX 模块实时接收来自指令寄存器（IR）提取的原始立即数字段作为输入。

动态扩展处理：根据控制单元产生的 ExtSel 信号类型，模块自动识别当前指令的需求，执行相应的有符号位扩展或无符号位扩展处理。

结果输出：经过逻辑转换后，模块将原始变长字段统一输出为标准的 32 位位宽数据，确保其能直接作为 ALU 或地址计算单元的操作数。

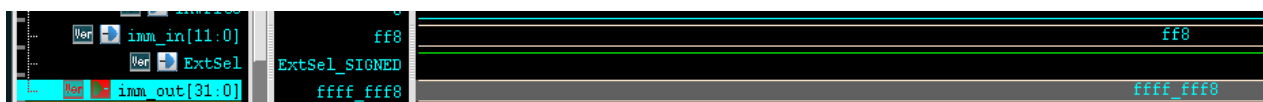


图 2-1-8 立即数拓展

2.2 R-TYPE 指令测试

指令分析:

- 机器码: 001105b3
- 汇编指令: add x11, x2, x1
- 预期结果: 执行 $x11 = x2 + x1$

执行过程分析:

IF（取指）阶段：

控制单元拉高 IRWrite 与 PCWrite 信号。在此阶段，指令寄存器（IR）准备锁存当前的指令码，同时 PC 准备在时钟上升沿更新为 NPC 的值，为下一条指令的提取做准备。

DAE (译码/执行) 阶段:

译码与控制：out_ins 完成更新，译码电路同步产出控制信号。系统将 ALUSrcA 置为 0，ALUSrcB 设为 ALUSrcB_B（选择寄存器操作数），并将 ALUOp 配置为加法运算模式。

操作数寻址：根据指令字段，系统自动选择 RD1（x2）与 RD2（x1）作为运算输入，并确定写入地址 WR 为 x11。

运算与缓冲：ALU 实时完成加法计算，运算结果通过一级 Flopr 锁存至 WD（写入数据总线）。同时，为了实现多周期重叠取指，本阶段提前拉高 InsMemRW，准备在下一周期更新 in ins。

WB（写回）阶段：

控制信号 RFWrite 被拉高，触发寄存器堆的写操作。仿真波形显示，WD 总线上的值（即 $0x0c + 0x08 = 0x14$ ）在时钟上升沿被准确写入地址 WR(x11) 中。

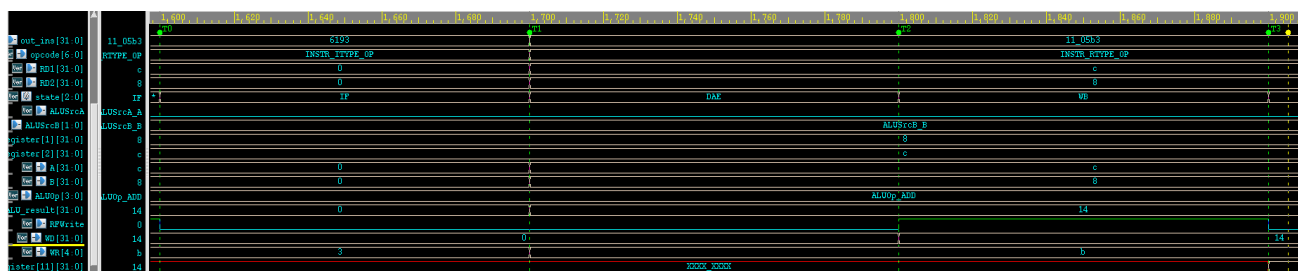


图 2-2-1 ADD 指令波形

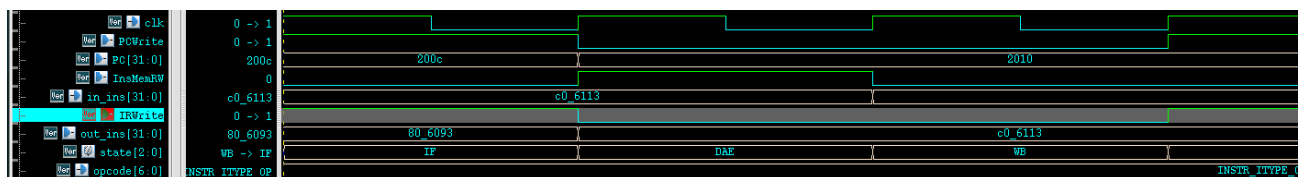


图 2-2-2 R-TYPE 指令 IR 与 PC 更新波形

指令分析:

- 机器码: 40110633
- 汇编指令: `sub x12, x2, x1`
- 预期结果: 执行 $x12 = x2 - x1$

仿真结果验证:

指令译码: 系统准确识别机器码中的功能位, 将 `ALUOp` 配置为减法模式。

操作数寻址: 根据指令字段, 自动从寄存器堆读取 `x2 (0x0c)` 与 `x1 (0x08)` 作为 `ALU` 的输入操作数。

运算写回: 如图所示, `ALU` 计算出的差值 `0x04` 正确产生并锁存至 `WD` 总线。在接下来的写回阶段, 该数值被准确存入目标寄存器 `x12` 地址处。

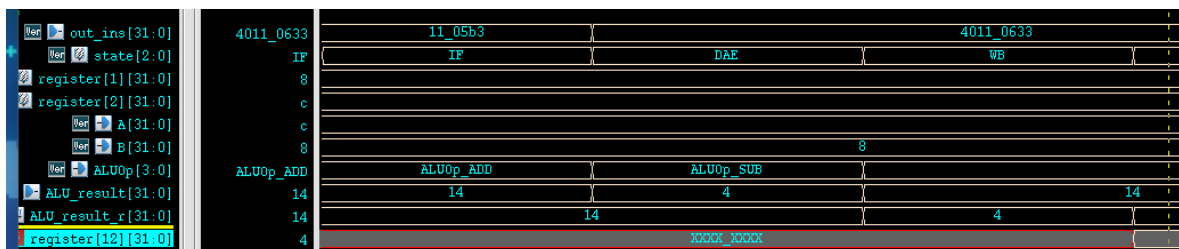


图 2-2-3 SUB 指令波形

指令分析:

- 机器码: 00316733
- 汇编指令: `or x14, x2, x3`
- 预期结果: 执行 $x14 = x2 | x3$

仿真结果验证:

指令译码: 系统识别功能字段后, 控制单元将 `ALUOp` 配置为按位或 (OR) 逻辑运算模式。

操作数寻址: 从寄存器堆准确读取 `x2 (0x0c)` 与 `x3 (0x00)` 作为输入操作数。

运算写回: 如图所示, `ALU` 完成按位或运算, 结果 `0x0c` 正确锁存至 `WD` 总线, 并最终在写回阶段更新至目标寄存器 `x14`。



图 2-2-4 OR 指令波形

指令分析:

- 机器码: 0020f7b3
- 汇编指令: and x15, x1, x2
- 预期结果: 执行 $x15 = x1 \& x2$

仿真结果验证:

指令译码: 控制单元根据指令字段将 ALUOp 配置为按位与 (AND) 逻辑运算模式。

操作数寻址: 系统从寄存器堆准确读取 x1 (0x08) 与 x2 (0x0c) 作为 ALU 的输入操作数。

运算写回: 如图所示, ALU 完成按位与运算, 结果 0x08 正确产生并锁存至 WD 总线, 随后在写回阶段更新至目标寄存器 x15。

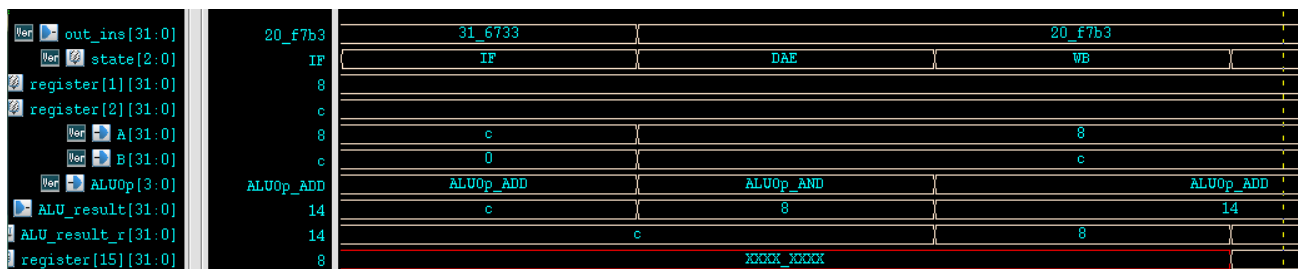


图 2-2-5 AND 指令波形

指令分析:

- 机器码: 001149b3
- 汇编指令: xor x19, x2, x1
- 预期结果: 执行 $x19 = x2 \wedge x1$

仿真结果验证:

指令译码: 控制单元通过识别机器码, 将 ALUOp 准确配置为按位异或 (XOR) 逻辑运算模式。

操作数寻址: 系统从寄存器堆读取 x2 (0x0c) 与 x1 (0x08) 作为输入操作数。

运算写回: 如图所示, ALU 完成异或运算, 结果 0x04 正确锁存至 WD 总线, 并最终在写回阶段更新至目标寄存器 x19。

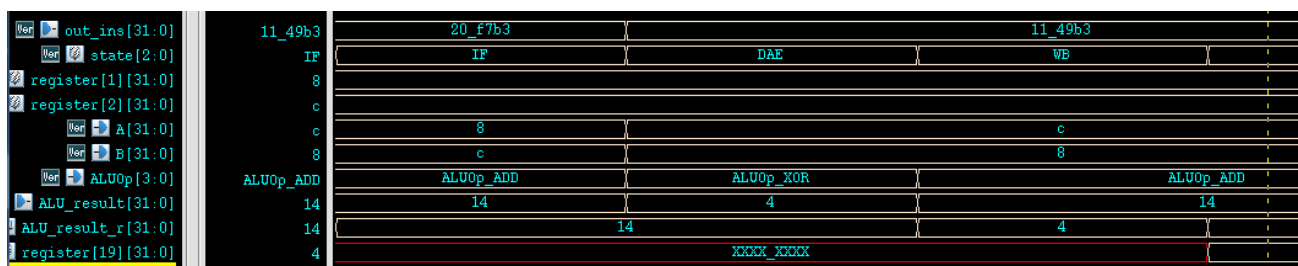


图 2-2-6 XOR 指令波形

指令分析:

- 机器码: 004113b3
- 汇编指令: sll x7, x2, x4
- 预期结果: 执行 $x7 = x2 \ll x4$

仿真结果验证:

指令译码: 控制单元通过解析功能字段, 将 ALUOp 准确配置为逻辑左移 (SLL) 运算模式。

操作数寻址: 系统从寄存器堆读取待移位值 x2 (0x0c) 与移位量 x4 (4) 作为 ALU 的输入操作数。

运算写回: 如图所示, ALU 完成左移运算, 结果 0xc0 正确锁存至 WD 总线, 并最终在写回阶段更新至目标寄存器 x7。

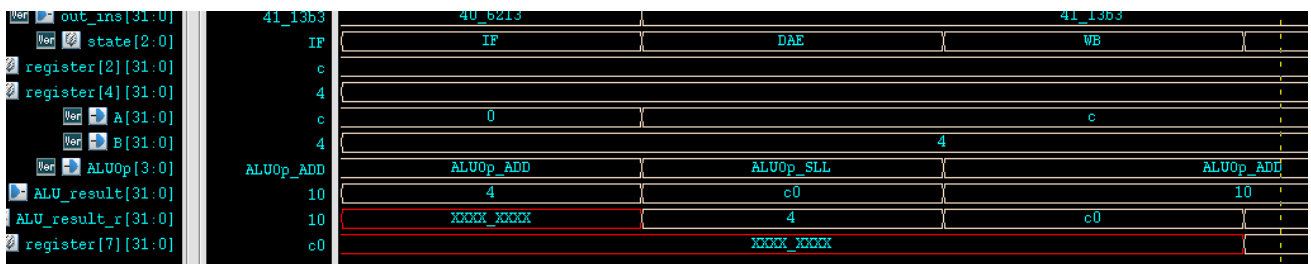


图 2-2-7 SLL 指令波形

指令分析:

- 机器码: 0047d333
- 汇编指令: srl x6, x15, x4
- 预期结果: 执行 $x6 = x15 \gg x4$ (逻辑右移)

仿真结果验证:

指令译码: 控制单元通过解析功能字段, 将 ALUOp 准确配置为逻辑右移 (SRL) 运算模式。

操作数寻址: 系统从寄存器堆读取待移位值 x15 (0x80) 与移位量 x4 (4) 作为 ALU 的输入操作数。

运算写回: 如图所示, ALU 完成逻辑右移运算, 结果 0x08 正确锁存至 WD 总线, 并最终在写回阶段更新至目标寄存器 x6。



图 2-2-8 SRL 指令波形

指令分析：

- 机器码：4047d2b3
- 汇编指令：sra x5, x15, x4
- 预期结果：执行 $x5 = x15 \ggg x4$ （算术右移）

仿真结果验证：

指令译码：控制单元准确识别功能字段，将 ALUOp 锁写为算术右移（SRA）模式。

操作数寻址：系统从寄存器堆读取待移位值 x15（当前值为 0x00000080）与移位量 x4（值为 4）作为 ALU 的输入操作数。

符号位扩展逻辑：在 32 位 CPU 架构中，算术右移需根据第 31 位（符号位）进行填充。由于 x15 的最高位为 0，右移 4 位后高位补 0，计算结果为 0x00000008。

运算写回：如图所示，ALU 运算结果 0x08 正确锁存至 WD 总线，并在写回阶段准确更新至目标寄存器 x5。



图 2-2-9 SRA 指令波形

2.3 I-TYPE 指令测试

指令分析：

- 机器码：00806093
- 汇编指令：ori x1, x0, 8
- 预期结果：执行 $x1 = x0 \mid 8$

执行过程分析：

IF（取指）阶段：

控制单元拉高 IRWrite 与 PCWrite 信号。在此阶段，指令存储器输出的指令码被锁定至指令寄存器，同时 PC 准备在时钟上升沿更新，指向下一条待取指令。

DAE（译码/执行）阶段：

译码与选择：out_ins 完成更新并由译码器解析。由于是 I-TYPE 指令，控制单元将 ALUSrcA 置为 0（选择寄存器数据），将 ALUSrcB 切换为 ALUSrcB_Imm（选择扩展后的立即数），并将 ALUOp 配置为按位或运算模式。

操作数处理：系统读取 RD1（x0，恒为 0）与扩展模块输出的立即数 Imm_out（值为 8）作为 ALU 输入。

运算与缓冲：ALU 实时完成逻辑或运算，计算结果通过一级 Flopr 同步锁存至 ALU_result_r 中。同时，拉高 InsMemRW 信号，为下一条指令的预取做准备。

WB（写回）阶段：

控制单元拉高 RFWrite 信号。在当前周期结束、下一个时钟上升沿到来时，保存在 ALU_result_r 中的计算结果（0x08）被正式写入目标寄存器 WR（x1）的地址空间中。



图 2-3-1 ORI 指令波形



图 2-3-2 I-TYPE 指令 IR 与 PC 更新波形

指令分析：

- 机器码：00110693
- 汇编指令：addi x13, x2, 1
- 预期结果：执行 $x13 = x2 + 1$

仿真结果验证：

指令译码与扩展：系统识别为 I-TYPE 加法指令，扩展模块（EX）将立即数字段正确扩展为 32 位数值 1。

操作数选择：ALUSrcB 切换至 ALUSrcB_Imm 路径，将寄存器 x2 的值（0x0c）与立即数（0x01）送入 ALU 运算端。

运算写回：仿真波形显示，ALU 计算结果 0x0d 经过锁存后，在写回阶段准确存入目标寄存器 x13。

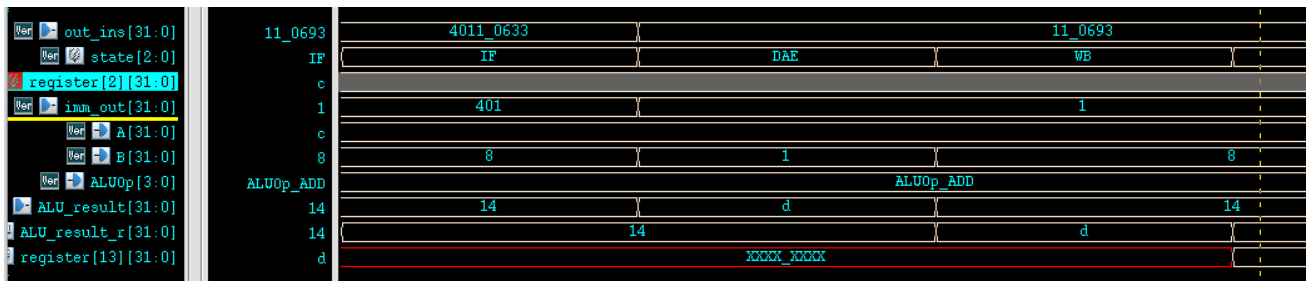


图 2-3-3 ADDI

2.4 SW 指令测试

指令分析：

- 机器码：fe40ae23
- 汇编指令：sw x4, -4(x1)
- 预期结果：将寄存器 x4 的数值写入到以 x1 - 4 为地址的存储单元中。

执行过程分析：

IF（取指）阶段：

控制单元拉高 IRWrite 与 PCWrite 信号。指令寄存器锁存当前的机器码，同时 PC 步进至下一条指令地址，完成取指操作。

DAE（译码/执行）阶段：

地址计算：译码器识别出 S-TYPE 指令格式。控制单元将 ALUSrcA 置为 0，ALUSrcB 切换至 ALUSrcB_Imm。扩展模块将立即数字段解析为有符号数 -4，并与 RD1（x1）的值相加。

结果缓冲：ALU 计算出的有效地址经过一级 Flopr 锁存为 ALU_result_r。与此同时，InsMemRW 拉高，确保下一条指令的预取逻辑正常运行。

MEM（访存）阶段：

存储写入：控制单元拉高存储器写使能信号 DMCtrl。此时，系统将 ALU_result_r 作为存储器访问地址（本例中 $x1 - 4 = 0x04$ ），并提取寄存器 RD2（x4）中的数据作为待写入值。

地址对齐：在实际存储映射中，地址 0x04 经过字对齐处理（截断低位偏移），准确指向存储

矩阵的相应字单元。

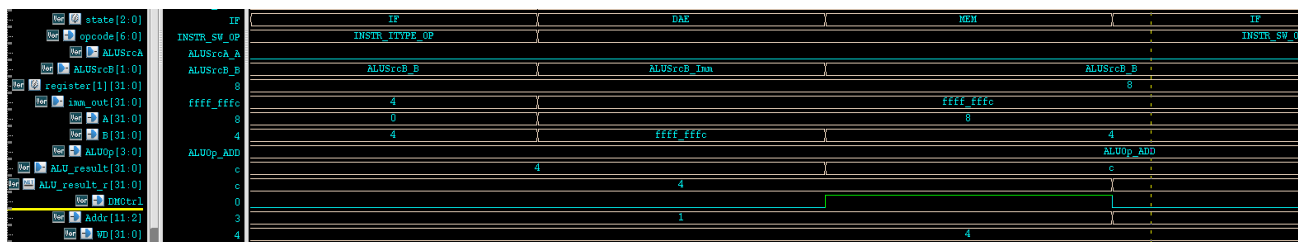


图 2-4-1 SW 指令波形



图 2-4-2 SW 指令 IR 与 PC 更新波形

2.5 LW 指令测试

指令分析：

- 机器码：ff80a283
- 汇编指令：lw x5, -8(x1)
- 预期结果：从内存地址 x1 - 8 处读取 32 位数据，并将其写入寄存器 x5。

执行过程分析：

IF（取指）阶段：

控制单元拉高 IRWrite 与 PCWrite 信号。在此阶段，当前指令被锁存至 IR，同时 PC 准备更新以指向后续指令。

DAE（译码/执行）阶段：

地址计算：译码完成后，控制单元将 ALUSrcA 置为 0，ALUSrcB 切换至 ALUSrcB_Imm。

扩展模块（EX）将立即数解析为有符号数 -8，并与 RD1（x1）相加计算有效地址。

结果缓冲：ALU 的加法计算结果经过一级 Flopr 锁存至 ALU_result_r（注：图中逻辑通常在此阶段确定访存地址）。同时拉高 InsMemRW 准备下一轮取指。

MEM（访存）阶段：

读取控制：控制单元将 DMCtrl 置为 0（读模式），以 ALU_result_r 作为地址从数据存储器（DM）中检索数据。

数据暂存：从指定内存地址读出的数据存入读数据寄存器 RD 中，等待写回。

WB（写回）阶段：

路径切换：控制单元将 WDSel 切换为 WDSel_FromMEM，使写入数据总线 WD 指向存储器读取的结果。

同步写回：拉高 RFWrite 使能信号。在下一个时钟上升沿，内存数据被正式写入目标寄存器 WR (x5) 中。

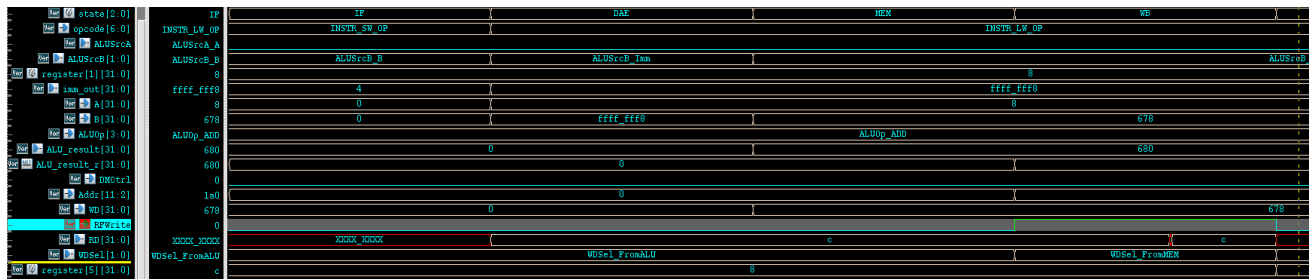


图 2-5-1 LW 指令波形



图 2-5-2 LW 指令 IR 与 PC 更新波形

2.6 B-TYPE 指令测试

指令分析：

- 机器码：fe519ce3
- 汇编指令：bne x3, x5, -8
- 预期结果：比较寄存器 x3 与 x5 的值。若两者不相等，则程序计数器 PC 跳转至当前地址偏移 -8 字节处；否则，顺序执行下一条指令。

执行过程分析：

IF（取指）阶段：

控制单元拉高 IRWrite 信号以锁存指令。同时，硬件逻辑预先对指令格式进行初步判定。由于识别到当前指令属于 B-TYPE 分支类型，为了防止在条件确定前错误的自增 PC，系统在此阶段将 PCWrite 置为 0，挂起自动递增逻辑。

DAE（译码/执行/跳转计算）阶段：

条件判定：译码完成后，控制单元将 ALUSrcA 置为 0，ALUSrcB 切换至 ALUSrcB_B（选择寄存器操作数）。ALU 执行减法运算 (x3 - x5)，并通过内部的 zero 标志位判断两者是否相等。

跳转决策：若 zero 信号为 0（表示不相等），分支预测成功。此时控制单元赋值 NPCOp 以

选择跳转目标地址（当前 PC + 偏移量），并重新拉高 PCWrite，强行将计算后的跳转地址更新至 PC 寄存器。

指令预取：同时拉高 InsMemRW 信号，确保在跳转目标确定后，能够及时更新 in_ins 以提取目标位置的指令。

WB（指令更新）阶段：

在该周期内，系统根据上一阶段确定的 PC 值完成指令存储器的读取，更新 in_ins 进入下一个取指循环。

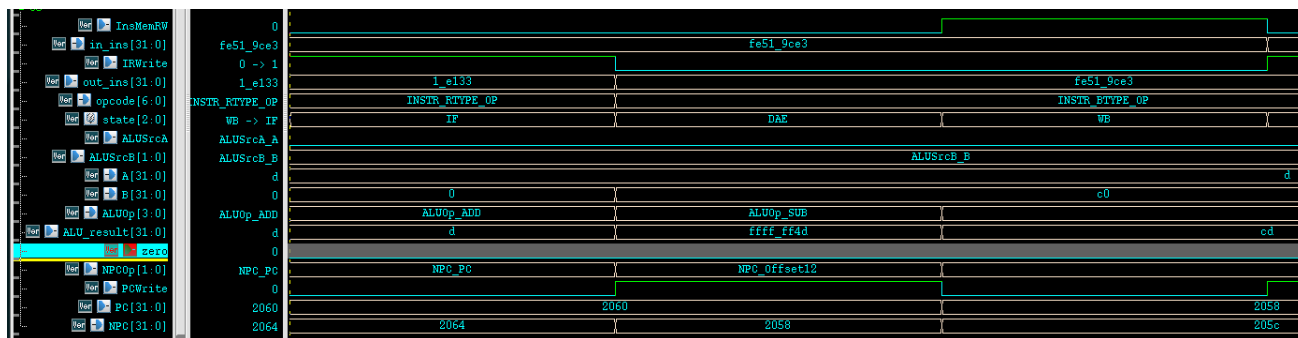


图 2-6-1 BNE 指令满足条件跳转波形

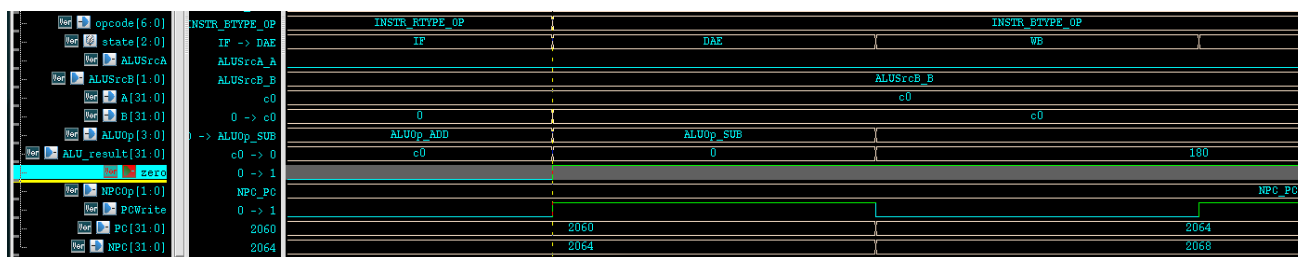


图 2-6-2 BNE 指令不满足条件波形

指令分析：

- 机器码：01cd8863
- 汇编指令：beq x27, x28, 16
- 预期结果：若 $x27 == x28$ ，则 $PC = PC + 16$ 。

仿真结果验证：

条件判定：系统读取寄存器 x27 与 x28 的值。仿真波形显示两者当前值均为 0xab，满足相等条件。

跳转执行：由于比较结果为“真”，控制单元通过 NPCOp 选择分支目标地址。PC 成功由当前地址 0x2084 跳转至目标地址 0x2094（即偏移 0x10 字节）。

时序响应：在 DAE 阶段完成地址计算后，PCWrite 重新拉高，确保在下一个周期从跳转后的新地址开始取指。

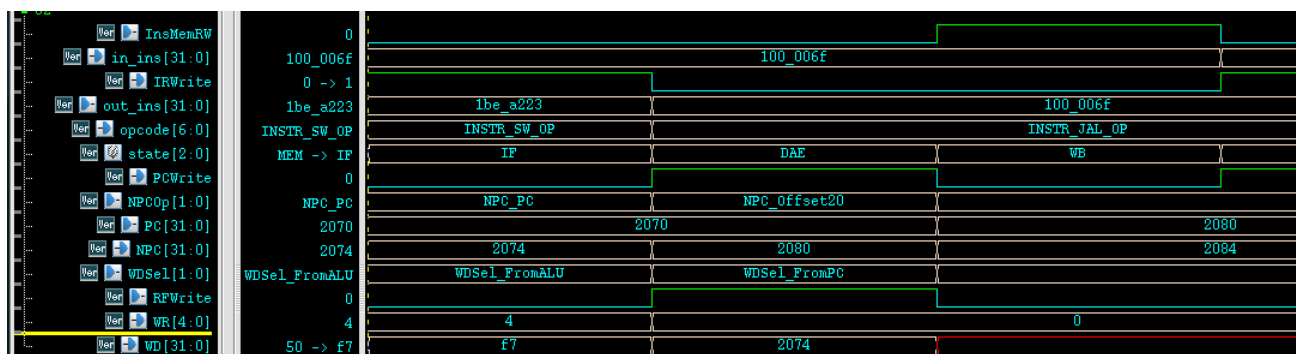


图 2-7-1 JALR 指令波形

2.8 JALR 指令测试

指令分析：

- 机器码：024302E7
- 汇编指令：jalr x5, x6, 36
- 预期结果：程序跳转至寄存器 x6 的值加上立即数偏移量 36 (0x24) 的地址；同时将当前指令的返回地址（PC + 4）保存至目标寄存器 x5。

执行过程分析：

IF（取指）阶段：

控制单元拉高 IRWrite 信号以锁存当前指令。硬件逻辑识别出 JALR 指令类型，为了确保 PC 跳转地址的准确性，在此阶段将 PCWrite 置为 0，挂起自动递增逻辑。

DAE（译码/执行/跳转写回）阶段：

跳转目标计算：译码完成后，控制单元将 ALUSrcA 置为 0（选择寄存器 x6），ALUSrcB 切换至 ALUSrcB_Imm（选择立即数 36）。ALU 完成加法运算，计算出目标地址。

PC 更新：控制单元配置 NPCOp 指向 ALU 的计算结果，并重新拉高 PCWrite。仿真显示，程序成功跳转至目标地址 0x2024（0x2000 + 0x0024）。

返回地址保存：控制单元将 WDSel 切换为 WD_FromPC 模式，将当前指令地址加上 4 字节的偏移量（0x2018 + 0x0004 = 0x201C）作为写入数据。

寄存器写回：拉高 RFWrite 信号，将计算出的返回地址 0x201C 正式写入目标寄存器 x5。

WB（指令更新）阶段：

拉高 InsMemRW 信号，系统根据跳转后的新 PC 值（0x2024）从存储器中提取下一条指令，完成控制流的平滑切换。

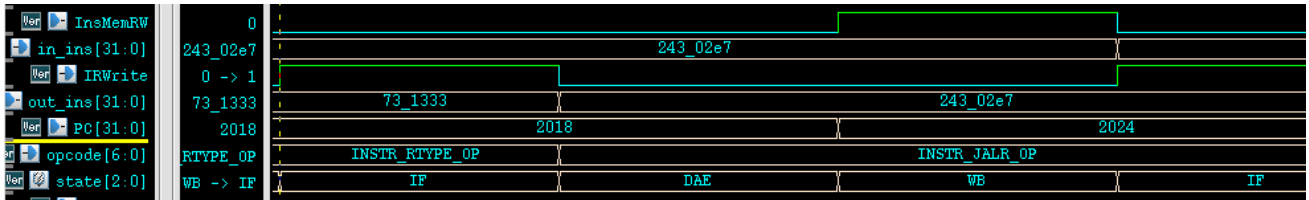


图 2-8-1 JALR 指令 IR 与 PC 模块波形

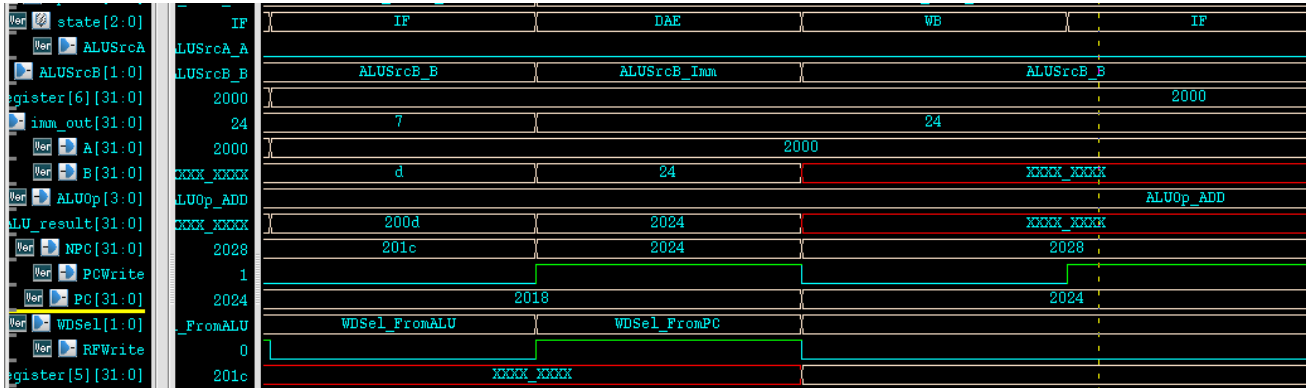


图 2-8-1 JALR 指令 ALU 模块波形

AI 工具使用声明

本报告在撰写过程中使用了人工智能辅助技术进行文档优化，具体声明如下：

(a)使用工具：Gemini 3 Flash。

(b)使用场景与用途：

文档格式及语言润色：利用 AI 优化技术分析段落的语言逻辑性、专业术语表达及文档格式。

技术描述标准化：辅助完善 R-TYPE、I-TYPE、S-TYPE、B-TYPE 及 J-TYPE 等各类 RISC-V 指令在多周期 CPU 架构下的数据通路描述，确保文档叙述风格的一致性。

测试指令集辅助生成：利用 AI 辅助构建针对 RISC-V 指令集的测试案例，通过生成特定的机器码序列与汇编指令组合，验证 CPU 对各类指令数据通路处理的覆盖率与准确性。

(c)内容占比说明：AI 优化内容约占全文总篇幅的 20%。报告中的仿真波形分析、核心电路设计及原始实验数据均由团队实验并记录。